

## Keith Framework - Databasable

### Introduction

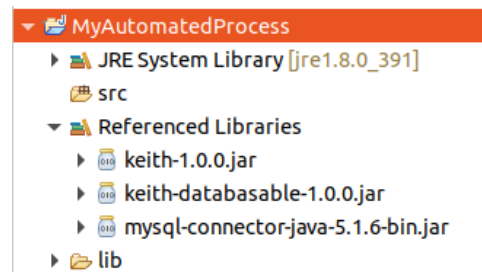
The Databasable enhancement for the Keith framework is a way to provide the common logic for accessing a database using the JDBC/DAO pattern. Only the basics for getting a database connection, closing resources, and logging warnings are provided. This allows implementing programmers the freedom to incorporate their own logic throughout the persistence process.

If you are not familiar with the Keith framework, then please review the documentation before continuing. If you are interested in writing an enhancement yourself, then please review the Keith Steppable documentation, as well as the source for the Steppable and Databasable enhancements. This document addresses implementing the IDatabasable interface in an automated process.

### Implementation

First, create a new Java project and add the keith-[version].jar. Then, add the keith-databasable-[version].jar. Finally, add the database connector jar(s) and any dependencies. Insure all are on the automated process classpath.

The [version] place holder should be replaced with the jar version such as 1.0.0.



Update the code to use the enhancement:

- Implement the interface in your class:

```
public class MyAutomatedProcess extends AbstractApplication
    implements IDatabasable {
```
- This enhancement uses Java util logging so include a class variable:

```
private static final Logger logger =
    Logger.getLogger("com.myautomatedprocess.MyAutomatedProcess");
```
- Add the contractual method from the interface:

```
@Override
public DatabaseProperties[] getDatabaseProperties() {
    DatabaseProperties[] models = new DatabaseProperties[1];
    models[0] = new DatabaseProperties("mysql");
    models[0].setDriver("com.mysql.jdbc.Driver");
    models[0].setUrl("jdbc:mysql://localhost:3306/mysql");
    models[0].setPassword("");
    models[0].setUsername("");
    return models;
}
```

## Keith Framework - Databasable

- Note the `getDatabaseProperties()` method supports multiple database definitions allowing an automated process to work with more than one database. This logic is dependent on unique IDs for each model.
- Add code using the enhancement:

```
@Override
protected void process() throws Exception {
    IDatabasable.validate(this);
}

@Override
protected void run() throws Exception {
    DatabaseAccess access = new DatabaseAccess("mysql");
    Connection conn = access.getConnection();
    PreparedStatement pstmt = conn.prepareStatement(
        "SELECT * FROM time_zone_name");
    ResultSet rs = pstmt.executeQuery();
    while (rs.next()) {
        System.out.println(rs.getString(1));
    }
    DatabaseAccess.destroy(conn, pstmt, rs, logger);
}
```

## Summary

Following the spirit of the Keith framework, an enhancement should support a common feature using as little code as possible. In other words, abstract the concept as much as possible to allow programmers creative freedom.

The IDatabasable enhancement makes decisions for functionality, yet remains simple. For instance, the primary concern for registering multiple databases is for the database ID (the value passed to the constructor) to be unique. Validation is embedded to insure compliance. Otherwise, all setup logic, including initializing the driver class, is performed by the enhancement. Additionally, the enhancement does not attempt to include connection pool logic, batch processes, or transactions. These features are typically specific to the individual job or preference for a 3<sup>rd</sup> party API, and left untouched by this code.

Possible enhancements may not be apparent at first, but as with other styles of applications, any time an operation is repeated in multiple automated processes, it may be useful to convert the operation to an API, then adapt the API to be used as part of the Keith framework.